



Trinity Power, Inc.
(512) 992-8657

Application Note AN-2 (rev 1.27)

for the

TPI Signal Generator

covering

User Command Structure

for TPI-1001, TPI-1002, & TPI-1005

unit firmware version 1.077

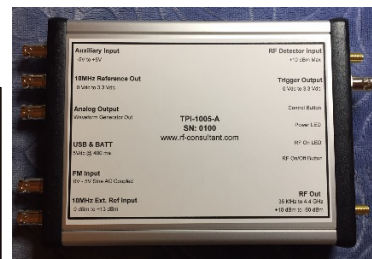
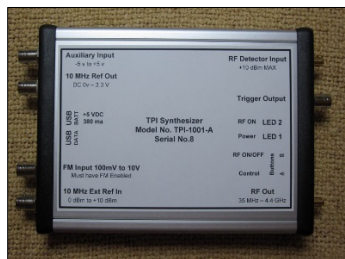


Table of Contents

1 Introduction	6
1.1 USB Interface	6
1.2 Packet Structure	6
1.3 Packet Framing	7
1.4 Traffic Control	7
1.5 ASCII Communications	7
2 Commands	8
2.1 User Control	8
2.2 Read Model Number	9
2.3 Read Serial Number	9
2.4 Read Hardware Version.....	9
2.5 Read Unit Firmware Version.....	9
2.6 Read Minimum TPI-Link Version Required.....	10
2.7 Read Supply Voltages.....	11
2.8 Current Unit State	12
2.9 Frequency	13
2.10 RF Output Level	13
2.11 RF Output On/Off	14
2.12 Read Detector Level (TPI-1001 & TPI-1005 only)	15
2.13 Detector On/Off (TPI-1001 & TPI-1005 only)	16
2.14 Reference Source.....	16
2.15 Reference Output On/Off.....	17
2.16 Read Auxiliary Input Voltage (TPI-1001 & TPI-1005 only)	17
2.17 Auxiliary Input Threshold (TPI-1001 & TPI-1005 only).....	18

2.18 Auxiliary Input Greater/Less Than (TPI-1001 & TPI-1005 only)	19
2.19 Auxiliary Input Action (TPI-1001 & TPI-1005 only)	20
2.20 Trigger Output Source (TPI-1001 & TPI-1005 only)	21
2.21 Invert Trigger Output (TPI-1001 & TPI-1005 only)	22
2.22 Frequency Presets	23
2.23 Level Presets	24
2.24 Beep	24
2.25 Control Button Function	25
2.26 Mute Till Lock Detect	26
2.27 Frequency Increment	26
2.28 Level Increment	27
2.29 Match Frequency	27
2.30 Select Frequency Preset	28
2.31 Select Level Preset	29
2.32 Save Current Sate as Default	30
2.33 Increment Frequency	31
2.34 Increment Level	31
2.35 PLL Automatic Reporting	32
2.36 Read PLL Lock	32
2.37 Read Running Time	33
2.38 Square Wave Modulation Parameters	34
2.39 Scan Parameters	35
2.40 Control Script	36
2.40.1 Read End of Script Step Number	37
2.40.2 Write a Script Step	37
2.40.3 Read a Script Step	38

2.40.4 Write the Control Script Name	38
2.40.5 Read the Control Script Name.....	39
2.40.6 Set the Control Script Run Control	39
2.40.7 Read the Control Script Run Control.....	40
2.40.8 Current Control Script Command.....	40
2.40.9 Read Control Script Frequency.....	41
2.40.10 Set Control Script Frequency	41
2.40.11 Read Control Script Power Level.....	42
2.40.12 Set Control Script Power Level	42
2.41 Read ADF4351 Registers.....	43
2.42 Set ADF4351 Registers	44
2.43 VCO Control	45
2.44 Auxiliary Level Control (TPI-1001 & TPI-1005 only)	46
2.45 Auxiliary Frequency Control (TPI-1001 & TPI-1005 only)	47
2.46 Frequency List (TPI-1001 & TPI-1005 only)	48
2.47 FM Source (TPI-1005 only)	50
2.48 FM Method (TPI-1005 only).....	51
2.49 Amplifier Source (TPI-1005 only).....	52
2.50 Waveform Generator (TPI-1005 only)	53
2.51 Amplifier Gain (TPI-1005 only)	54
2.52 Analyzer (TPI-1001 and TPI-1005 only)	55
2.52.1 Analyzer Parameters	55
2.52.2 Control Analyzer.....	56
2.52.3 Analyzer Data.....	57
2.52.4 Analyzer Stopped	57
2.53 Trigger Output (TPI-1001 & TPI-1005 only).....	58

2.54 ADC Average.....	59
2.55 FM Method (TPI-1006 only).....	60
2.56 Error Notification	61
3 C# Code Examples	62
3.1 Establish the COM Port	62
3.2 Send a Packet	62
3.3 Read Bytes From the COM Port	63
4 ASCII Commands	64
4.1 RF output on/off.....	64
4.2 Frequency	65
4.3 Output level	65
4.4 Detector level	65
5 SCPI Commands	66
5.1 *RST.....	66
5.2 [:]SOURce:FREQuency <num>	66
5.3 [:]SOURce:FREQuency?	67
5.4 [:]SOURce:POWer <integer>	67
5.5 [:]SOURce:POWer?	67
5.6 [:]SOURce:OUTPut:STATe ON OFF	67
5.7 [:]SOURce:OUTPut:STATe?	67
6 Revisions	68

1 Introduction

This application note is a supplement to the TPI-1001, TPI-1002, and TPI-1005 signal generator user manual and describes the user accessible command structure. With this information, users may create their own user interface to communicate with the signal generator instead of using the supplied graphical user interface (GUI) TPI-Link. Note that some features of the signal generator are only available via TPI-Link. Unit firmware version 1.017 is the first version of firmware that supports most of these commands.

1.1 USB Interface

Some units have two USB ports. One of the ports on each unit is only used for supplemental power from a second power source such as a battery. The other USB port is used for both powering the unit and communication via the USB port of a computer. The TPI-1001 communication port is labeled “USB Data”, and the TPI-1002 communication port is labeled “USB”.

Internally the communication USB port is connected to an FTDI chip that acts as a USB to UART translator. Therefore, the user’s program accesses the unit as if it was a COM port. The driver installed in the computer maps the computer’s USB port to a virtual COM port. TPI-Link can be used to identify the COM port number in use. This can also be seen using the Windows Device Manager.

The user’s program must set the COM port parameters to the following...

- Baud rate: 3,000,000 (3 Mbit/sec)
- Data bits: 8
- Stop bits: 1
- Parity: None
- Handshake: Request to send

1.2 Packet Structure

Throughout this document byte values will be represented in hexadecimal notation with “0x” preceding the value unless otherwise noted as a decimal value. For example, the decimal value 123 is equivalent to 0x7B.

All communications to or from the unit are in the form of a defined packet with a header, a body, and a checksum. The header consists of four bytes...

- 1st byte: 0xAA (decimal 170) — First qualifier byte
- 2nd byte: 0x55 (decimal 85) — Second qualifier byte
- 3rd byte: High order byte of 16 bit integer representing number of bytes in body
- 4th byte: Low order byte of 16 bit integer representing number of bytes in body

The body of the packet consists of a minimum of two bytes. See the next section for a detailed list of the various commands available.

The body is followed by a one-byte checksum that is the ones' complement of the 8-bit sum of all preceding bytes in the packet excluding the two qualifier bytes. In other words, it is 0xFF minus the sum of all bytes beginning with the 3rd byte of the header through the last byte of the body. For example, if the body of a packet sent to the unit consisted of the two bytes 0x07, 0x02, the entire packet would be (header, body, checksum)...

0xAA, 0x55, 0x00, 0x02, 0x07, 0x02, 0xF4

1.3 Packet Framing

The two qualifier bytes (0xAA, 0x55) provide a means of identifying the beginning of a packet. If the user's program receives an incorrect checksum or somehow gets out of sync with the expected number of bytes in the body of a packet, the program should look for the two-byte qualifier sequence to identify the beginning of the next packet. Note that it is possible that the same two-byte sequence may be contained within the body of a packet. Therefore, once the qualifier sequence has been received, the program must not interpret any subsequent identical two-byte sequences as packet qualifiers until the expected number of bytes has been received.

1.4 Traffic Control

When the unit receives a packet, it will interpret the packet and execute the command as quickly as possible. The unit will then reply with the data requested or a simple acknowledgement. If the user sends another packet before a reply is received from the prior packet, the unit will buffer the new packet for processing after the prior one is finished. The receive buffer is several hundred bytes, which can permit many packets to be sent before the first one is fully processed. However, it is recommended that the user's program does not send another packet until a reply has been received from the previous packet. This will keep buffers from overflowing and makes program debugging much easier.

1.5 ASCII Communications

Instead of following the packet protocol outlined above, beginning with firmware version 1.065, the basic parameters can be read and written using simple ASCII strings. See section 4.

2 Commands

This section describes the various command packets. Only the body of each packet is shown. It must be understood that each of the following is preceded by the four-byte header and followed by the one-byte checksum described in section 1.2.

The first byte of the body indicates whether the command is a read or write command. A first byte of **0x07** indicates a read command, and **0x08** indicates a write command.

2.1 User Control

User control must first be enabled. Send the following once.

0x08, 0x01

The full packet including the header and checksum is...

0xAA, 0x55, 0x00, 0x02, 0x08, 0x01, 0xF4

This checksum was calculated as follows...

0x00 + 0x02 + 0x08 + 0x01 = 0x0B

The ones' complement of 0x0B is **0xF4**

The unit will respond with the same packet...

0x08, 0x01

User control is now enabled. The unit is ready to respond to any of the following commands. The user control state may be read at any time by sending the following bytes...

0x07, 0x01

The unit will respond with three bytes...

0x07, 0x01, n

...where **n** is **0x01** if user control is enabled and **0x00** if disabled.

User control defaults to **0x00** upon power up of the unit.

2.2 Read Model Number

To read the model number of the unit, send...

0x07, 0x02

The unit will respond with 18 bytes...

0x07, 0x02, n₀...n₁₅

...where n₀...n₁₅ is a 16-byte ASCII string (not null terminated) representing the model number of the unit.

2.3 Read Serial Number

To read the serial number of the unit, send...

0x07, 0x03

The unit will respond with 18 bytes...

0x07, 0x03, n₀...n₁₅

...where n₀...n₁₅ is a 16-byte ASCII string (not null terminated) representing the serial number of the unit.

2.4 Read Hardware Version

To read the hardware version of the unit, send...

0x07, 0x04

The unit will respond with 18 bytes...

0x07, 0x04, n₀...n₁₅

...where n₀...n₁₅ is a 16-byte ASCII string (not null terminated) representing the hardware version of the unit.

2.5 Read Unit Firmware Version

To read the firmware version of the unit, send...

0x07, 0x05

The unit will respond with 18 bytes...

0x07, 0x05, n₀...n₁₅

...where n₀...n₁₅ is a 16-byte ASCII string (not null terminated) representing the firmware version of the unit.

2.6 Read Minimum TPI-Link Version Required

To read the minimum version of TPI-Link that is compatible with the firmware in the unit, send...

0x07, 0x06

The unit will respond with 10 bytes...

0x07, 0x06, n₀...n₇

...where...

n₀...n₁ is a 16-bit integer (LSB first) = major

n₂...n₃ is a 16-bit integer (LSB first) = minor

n₄...n₅ is a 16-bit integer (LSB first) = build

n₆...n₇ is a 16-bit integer (LSB first) = revision

The entire required GUI version is represented by “major.minor.build.revision”. TPI-Link indicates its version by displaying only “major.minor” with minor represented as a three digit value. Therefore, a major value of “1” and a minor value of “23” will be displayed in TPI-Link as version 1.023.

Note: TPI-Link reads this information and compares it to the running version of TPI-Link to determine if the user should be warned about TPI-Link being incompatible with the current unit firmware. A custom user interface using the commands in this document should not be concerned about this command. Of greater concern to a custom user interface is if the user command structure has changed in any way. Any revisions to the user command structure in future versions of the unit firmware will be noted in revisions to this document.

2.7 Read Supply Voltages

To read the various supply voltages in the unit, send...

0x07, 0x07

The unit will respond with 26 bytes...

0x07, 0x07, n0...n23

...where...

n0...n3 is a 32-bit floating point value (IEEE-754, LSB first) = RF supply

n4...n7 is a 32-bit floating point value (IEEE-754, LSB first) = VCO supply

n8...n11 is a 32-bit floating point value (IEEE-754, LSB first) = MCU supply

n12...n15 is a 32-bit floating point value (IEEE-754, LSB first) = OSC supply

n16...n19 is a 32-bit floating point value (IEEE-754, LSB first) = PA supply

n20...n23 is a 32-bit floating point value (IEEE-754, LSB first) = DET supply*

*DET supply (TPI-1001); USB supply (TPI-1002)

2.8 Current Unit State

To read the current operational state of the unit, send...

0x07, 0x08

The unit will respond with 4 bytes...

0x07, 0x08, n₀, n₁

...where...

n ₀	Unit State	n ₁
0x00	Generator	Ignore value of n ₁
0x01	Modulation: square wave running	n ₁ >0 means RF output auto on/off
0x02	Modulation: beacon running	
0x03	Modulation: script running	
0x04	Scanning	Read n ₁ : 1=scanning; 2=paused Write n ₁ : 0=stop; 1=scan; 2=pause
0x05	Control script running	Read n ₁ : 1=script waiting Write n ₁ : 0=stop; 1=start; 2=continue

To set the current operational state of the unit, send...

0x08, 0x08, n₀, n₁

The unit will respond with 2 bytes...

0x08, 0x08

It is your responsibility to ensure that any state other than generator is terminated before initiating a new state. For example, if the unit is scanning (n₀ = 4), you must first stop the scanning by sending 0x08, 0x08, 0x04, 0x00 before starting another state, such as modulation or control script. If not, unpredicted behavior will result. TPI-Link enforces this requirement.

2.9 Frequency

To read the current frequency, send...

0x07, 0x09

The unit will respond with 6 bytes...

0x07, 0x09, n₀...n₃

...where n₀...n₃ is a 32-bit integer (LSB first) representing the frequency in kHz.

To set the frequency, send...

0x08, 0x09, n₀...n₃

...where n₀...n₃ is a 32-bit integer (LSB first) representing the frequency in kHz. The value must be between 35000 and 4400000 inclusive.

The unit will respond with 2 bytes...

0x08, 0x09

2.10 RF Output Level

To read the current output level, send...

0x07, 0x0A

The unit will respond with 3 bytes...

0x07, 0x0A, n

...where n is a signed byte representing the output level in dBm.

To set the level, send...

0x08, 0x0A, n

...where n is a signed byte representing the desired output level in dBm. If the requested output level is outside the available range, the output will be set to the maximum or minimum as appropriate.

The unit will respond with 3 bytes...

0x08, 0x0A, n

...where n is a signed byte representing the new output level in dBm.

2.11 RF Output On/Off

To read the current state of the output, send...

0x07, 0x0B

The unit will respond with 3 bytes...

0x07, 0x0B, n

...where **n = 0** when the output is off, and **n = 1** when the output is on.

To control the output, send...

0x08, 0x0B, n

...where **n = 0** to turn the output off, and **n = 1** to turn the output on.

The unit will respond with 2 bytes...

0x08, 0x0B

2.12 Read Detector Level (TPI-1001 & TPI-1005 only)

To read the detector level, send...

0x07, 0x0C

The unit will respond with 7 bytes...

0x07, 0x0C, n0...n4

...where...

n0...n3 is a 32-bit floating point value (IEEE-754, LSB first) = input level in dBm^{Note 2}

n4 = 0 means input level is within detection range

n4 = 1 means input level is >= maximum detection level

n4 = 2 means input level is <= minimum detection level

Notes:

¹ If this command is sent to a TPI-1002, an error code will be triggered because the TPI-1002 does not have a detector input.

² If the detector is not turned on, the returned value must be disregarded.

³ The detector is a broadband detector and so indicates the total RF power present across the spectrum. The detector's output has been calibrated at various frequencies and levels. Therefore, for highest accuracy, the unit needs to know the frequency of the signal presented to the RF detector input. It assumes that the frequency is the same as the frequency generated for the RF output. This feature accommodates the analyzer in TPI-Link. Therefore, if you are using the detector input as a generic RF power meter, it is important to set the RF output frequency to that of the frequency being measured for highest accuracy of the measurement.

2.13 Detector On/Off (TPI-1001 & TPI-1005 only)

To read the current state of the detector, send...

0x07, 0x0D

The unit will respond with 3 bytes...

0x07, 0x0D, n

...where **n = 0** when the detector is off, and **n = 1** when the detector is on.

To control the detector, send...

0x08, 0x0D, n

...where **n = 0** to turn the detector off, and **n = 1** to turn the detector on.

The unit will respond with 2 bytes...

0x08, 0x0D

2.14 Reference Source

To read the current reference source, send...

0x07, 0x0E

The unit will respond with 3 bytes...

0x07, 0x0E, n

...where **n = 0** when the reference source is internal, and **n = 1** when the reference source is external.

To select the reference source, send...

0x08, 0x0E, n

...where **n = 0** to select the internal reference, and **n = 1** to select the external reference.

The unit will respond with 2 bytes...

0x08, 0x0E

2.15 Reference Output On/Off

To read the current state of the reference output, send...

0x07, 0x0F

The unit will respond with 3 bytes...

0x07, 0x0F, n

...where **n = 0** when the reference output is off, and **n = 1** when the reference output is on.

To control the reference output, send...

0x08, 0x0F, n

...where **n = 0** to turn the reference output off, and **n = 1** to turn the reference output on.

The unit will respond with 2 bytes...

0x08, 0x0F

2.16 Read Auxiliary Input Voltage (TPI-1001 & TPI-1005 only)

To read the auxiliary input voltage, send...

0x07, 0x10

The unit will respond with 6 bytes...

0x07, 0x10, n₀...n₃

...where...

n₀...n₃ is a 32-bit floating point value (IEEE-754, LSB first) = auxiliary input in volts^{Notes 2, 3}

Notes:

¹ If this command is sent to a TPI-1002, an error code will be triggered because the TPI-1002 does not have an auxiliary input.

² The auxiliary input is internally biased to approximately 0.5 volts. Therefore, if the input is not connected to anything, you will read approximately 0.5 volts with this command.

³ The range of the auxiliary input is -5 volts to +5 volts.

2.17 Auxiliary Input Threshold (TPI-1001 & TPI-1005 only)

To read the auxiliary input threshold, send...

0x07, 0x11

The unit will respond with 6 bytes...

0x07, 0x11, n₀...n₃

...where...

n₀...n₃ is a 32-bit floating point value (IEEE-754, LSB first) = the auxiliary input threshold in volts
Note 2

To set the auxiliary input threshold, send...

0x08, 0x11, n₀...n₃

...where...

n₀...n₃ is a 32-bit floating point value (IEEE-754, LSB first) = the auxiliary input threshold in volts
Notes 1, 2

The unit will respond with 2 bytes...

0x08, 0x11

Notes:

¹ If this command is sent to a TPI-1002, an error code will be triggered because the TPI-1002 does not have an auxiliary input.

² The auxiliary input threshold has a range of -4 volts to +4 volts.

2.18 Auxiliary Input Greater/Less Than (TPI-1001 & TPI-1005 only)

To read the auxiliary input greater/less than, send...

0x07, 0x12

The unit will respond with 3 bytes...

0x07, 0x12, n

...where **n = 0** means the auxiliary input action will occur when the input falls below the threshold, and **n = 1** means the auxiliary input action will occur when the input rises above the threshold.

To set the auxiliary input greater/less than, send...

0x08, 0x12, n

...where **n = 0** means you wish the auxiliary input action to occur when the input falls below the threshold, and **n = 1** means you wish the auxiliary input action to occur when the input rises above the threshold.

The unit will respond with 2 bytes...

0x08, 0x12

2.19 Auxiliary Input Action (TPI-1001 & TPI-1005 only)

To read the current auxiliary input action, send...

0x07, 0x13

The unit will respond with 3 bytes...

0x07, 0x13, n

...where...

n	Auxiliary Input Action
0x00	None
0x01	RF output on while active
0x02	Turn RF output on
0x03	Turn RF output off
0x04	Increment frequency
0x05	Decrement frequency
0x06	Increment level
0x07	Decrement level
0x08	Mod: Square wave source
0x09	Mod: Square wave active
0x0A	Mod: Square wave start
0x0B	Mod: Square wave stop
0x0C	Mod: Beacon active
0x0D	Mod: Beacon start
0x0E	Mod: Beacon stop
0x0F	Mod: Script active
0x10	Mod: Script start
0x11	Mod: Script stop
0x12	Scan while active
0x13	Start scan
0x14	Stop scan
0x15	Pause scan
0x16	Script run active
0x17	Script start
0x18	Script stop
0x19	Script continue
0x1A	Auxiliary input controls level
0x1B	Auxiliary input controls freq
0x1C	Start analyzer
0x1D	Next frequency in list

To set the auxiliary input action, send...

0x08, 0x13, n

...where n equals the desired action from the list. The unit will respond with 2 bytes...

0x08, 0x13

2.20 Trigger Output Source (TPI-1001 & TPI-1005 only)

To read the current trigger output source, send...

0x07, 0x14

The unit will respond with 3 bytes...

0x07, 0x14, n

...where...

n	Trigger Output Source
0x00	None
0x01	Synth Lock Signal (MUXOUT)
0x02	RF Output State
0x03	Aux In > Threshold
0x04	Frequency = Match
0x05	Frequency > Match
0x06	Active while scanning (not paused or inter-scan)
0x07	Active while square wave "on"
0x08	Active while beacon "on"
0x09	Active while modulation script "on"
0x0A	Active during modulation script prefix
0x0B	Active during modulation script data
0x0C	Active during modulation script suffix
0x0D	Script controls trigger
0x0E	Detector level > level threshold
0x0F	Pulse at scan frequency step
0x10	Pulse at scan level step

See the user manual for a description of each source.

To set the trigger output source, send...

0x08, 0x14, n

...where n equals the desired source from the list.

The unit will respond with 2 bytes...

0x08, 0x14

2.21 Invert Trigger Output (TPI-1001 & TPI-1005 only)

To read the invert trigger output state, send...

0x07, 0x15

The unit will respond with 3 bytes...

0x07, 0x15, n

...where **n = 0** means the trigger output state is not inverted, and **n = 1** means the trigger output state is inverted.

To set the invert trigger output state, send...

0x08, 0x15, n

...where **n = 0** means you do not wish to invert the trigger output state, and **n = 1** means you wish to invert the trigger output state.

The unit will respond with 2 bytes...

0x08, 0x15

2.22 Frequency Presets

To read the twelve preset frequencies, send...

0x07, 0x16

The unit will respond with 50 bytes...

0x07, 0x16, n0...n47

...where...

n0...n3 is a 32-bit integer (LSB first) representing preset 1 frequency in kHz

n4...n7 is a 32-bit integer (LSB first) representing preset 2 frequency in kHz

n8...n11 is a 32-bit integer (LSB first) representing preset 3 frequency in kHz

n12...n15 is a 32-bit integer (LSB first) representing preset 4 frequency in kHz

n16...n19 is a 32-bit integer (LSB first) representing preset 5 frequency in kHz

n20...n23 is a 32-bit integer (LSB first) representing preset 6 frequency in kHz

n24...n27 is a 32-bit integer (LSB first) representing preset 7 frequency in kHz

n28...n31 is a 32-bit integer (LSB first) representing preset 8 frequency in kHz

n32...n35 is a 32-bit integer (LSB first) representing preset 9 frequency in kHz

n36...n39 is a 32-bit integer (LSB first) representing preset 10 frequency in kHz

n40...n43 is a 32-bit integer (LSB first) representing preset 11 frequency in kHz

n44...n47 is a 32-bit integer (LSB first) representing preset 12 frequency in kHz

To set the twelve preset frequencies, send...

0x08, 0x16, n0...n47

...where n0...n47 are the twelve preset frequencies as outlined above.

The unit will respond with 2 bytes...

0x08, 0x16

2.23 Level Presets

To read the twelve preset levels, send...

0x07, 0x17

The unit will respond with 14 bytes...

0x07, 0x17, n₀...n₁₁

...where...

- n₀ is a signed byte representing preset 1 level in dBm
- n₁ is a signed byte representing preset 2 level in dBm
- n₂ is a signed byte representing preset 3 level in dBm
- n₃ is a signed byte representing preset 4 level in dBm
- n₄ is a signed byte representing preset 5 level in dBm
- n₅ is a signed byte representing preset 6 level in dBm
- n₆ is a signed byte representing preset 7 level in dBm
- n₇ is a signed byte representing preset 8 level in dBm
- n₈ is a signed byte representing preset 9 level in dBm
- n₉ is a signed byte representing preset 10 level in dBm
- n₁₀ is a signed byte representing preset 11 level in dBm
- n₁₁ is a signed byte representing preset 12 level in dBm

To set the twelve preset levels, send...

0x08, 0x17, n₀...n₁₁

...where n₀...n₁₁ are the twelve preset levels as outlined above.

The unit will respond with 2 bytes...

0x08, 0x17

2.24 Beep

If a beep command is executed while a command script is running, the unit will send...

0x07, 0x18

This command is never requested by the user. When TPI-Link receives this command, it produces a short beep on the computer's audio output.

2.25 Control Button Function

To read the current control button function, send...

0x07, 0x19

The unit will respond with 3 bytes...

0x07, 0x19, n

...where...

n	Control Button Function
0x00	None
0x01	Square wave modulation
0x02	Beacon modulation
0x03	Modulation script
0x04	Scan
0x05	Control Script

See the user manual for a description of each function.

To set the control button function, send...

0x08, 0x19, n

...where n equals the desired function from the list.

The unit will respond with 2 bytes...

0x08, 0x19

2.26 Mute Till Lock Detect

To read the mute till lock detect (MTLD) state, send...

0x07, 0x1A

The unit will respond with 3 bytes...

0x07, 0x1A, n

...where **n = 0** means MTLD is disabled, and **n = 1** means MTLD is enabled.

To set the MTLD state, send...

0x08, 0x1A, n

...where **n = 0** means you wish disable MTLD, and **n = 1** means you wish enable MTLD.

The unit will respond with 2 bytes...

0x08, 0x1A

Note: MTLD should be disabled when using the FM input on the unit.

2.27 Frequency Increment

To read the current frequency increment, send...

0x07, 0x1B

The unit will respond with 6 bytes...

0x07, 0x1B, n₀...n₃

...where **n₀...n₃** is a 32-bit integer (LSB first) representing the frequency increment in kHz.

To set the frequency increment, send...

0x08, 0x1B, n₀...n₃

...where **n₀...n₃** is a 32-bit integer (LSB first) representing the frequency increment in kHz. The value must be between 1 and 4365000 inclusive.

The unit will respond with 2 bytes...

0x08, 0x1B

2.28 Level Increment

To read the current level increment, send...

0x07, 0x1C

The unit will respond with 3 bytes...

0x07, 0x1C, n

...where **n** is an unsigned byte representing the level increment in dBm.

To set the level increment, send...

0x08, 0x1C, n

...where **n** is an unsigned byte representing the desired level increment in dBm.

The unit will respond with 3 bytes...

0x08, 0x1C, n

Note: The level increment is always a positive value from 1 to 120 inclusive.

2.29 Match Frequency

To read the current match frequency, send...

0x07, 0x1D

The unit will respond with 6 bytes...

0x07, 0x1D, n₀...n₃

...where **n₀...n₃** is a 32-bit integer (LSB first) representing the match frequency in kHz.

To set the match frequency, send...

0x08, 0x1D, n₀...n₃

...where **n₀...n₃** is a 32-bit integer (LSB first) representing the match frequency in kHz. The value must be between 35000 and 4400000 inclusive.

The unit will respond with 2 bytes...

0x08, 0x1D

2.30 Select Frequency Preset

To select a frequency preset, send...

0x08, 0x1E, n

...where n is a value from 1 to 12 inclusive representing the desired preset.

The unit will respond with 6 bytes...

0x08, 0x1E, n₀...n₃

...where n₀...n₃ is a 32-bit integer (LSB first) representing the new frequency in kHz. If the value of n sent to the unit is not in the range of 1...12, the returned value of n₀...n₃ will be zero.

2.31 Select Level Preset

To select a level preset, send...

0x08, 0x1F, n

...where n is a value from 1 to 12 inclusive representing the desired preset.

The unit will respond with 3 bytes...

0x08, 0x1F, n

...where n is a signed byte representing the output level in dBm. If the value of n sent to the unit is not in the range of 1...12, the returned value of n will be 100 (decimal).

2.32 Save Current State as Default

To save the current state of the unit as the default, send...

0x08, 0x20

All the current parameter values will be saved in nonvolatile memory for use on power up. The current state of the unit will be saved, e.g., if the unit was scanning, it will begin scanning on power up.

The unit will respond with 3 bytes...

0x08, 0x20, n

...where **n** is zero if everything was saved with no errors, and one if there were any errors writing to nonvolatile memory.

2.33 Increment Frequency

To increment the frequency, send...

0x08, 0x21, n

...where **n** = 0 to increase the frequency by the frequency increment, and **n** = 1 to decrease the frequency by the frequency increment.

The unit will respond with 6 bytes...

0x08, 0x21, n₀...n₃

...where **n₀...n₃** is a 32-bit integer (LSB first) representing the new frequency in kHz. If the value of **n** sent to the unit is not in the range of 0...1, the frequency will not be changed.

2.34 Increment Level

To increment the level, send...

0x08, 0x22, n

...where **n** = 0 to increase the level by the level increment, and **n** = 1 to decrease the level by the level increment.

The unit will respond with 3 bytes...

0x08, 0x22, n

... where **n** is a signed byte representing the new output level in dBm. If the value of **n** sent to the unit is not in the range of 0...1, the level will not be changed.

2.35 PLL Automatic Reporting

To read the current state of the phase locked loop (PLL) automatic reporting, send...

0x07, 0x23

The unit will respond with 3 bytes...

0x07, 0x23, n

...where...

n	PLL Automatic Reporting
0x00	PLL lock will not be automatically reported
0x01	PLL lock will be reported whenever it changes
0x02	PLL lock will be reported when it changes, but not more frequently than every 0.25 seconds.

To set the automatic reporting, send...

0x08, 0x23, n

...where n equals the desired function from the list.

The unit will respond with 2 bytes...

0x08, 0x23

Note that the unit will always default to zero or “No automatic reporting” on power up. When automatic reporting is enabled (n=1 or n=2), the unit will automatically send...

0x07, 0x24, n

...as indicated in section 2.36 whenever the PLL lock changes.

2.36 Read PLL Lock

To read the current state of the PLL, send...

0x07, 0x24

The unit will respond with 3 bytes...

0x07, 0x24, n

...where n = 0 when the PLL is unlocked, and n = 1 when the PLL is locked.

2.37 Read Running Time

To read the time that the unit has been powered, send...

0x07, 0x25

The unit will respond with 7 bytes...

0x07, 0x25, n₀...n₄

...where...

n₀...n₁ is a 16-bit integer (LSB first) representing the number of days (0–65535 decimal)

n₂ is a byte representing the number of hours (0–23 decimal)

n₃ is a byte representing the number of minutes (0–59 decimal)

n₄ is a byte representing the number of seconds (0–59 decimal)

2.38 Square Wave Modulation Parameters

To read the square wave modulation parameters, send...

0x07, 0x26

The unit will respond with 35 bytes...

0x07, 0x26, n₀...n₃₂

...where...

- n₀...1 is a 16-bit integer (LSB first) = number of microseconds ON (10-16383)
- n₂...3 is a 16-bit integer (LSB first) = number of microseconds OFF (10-16383)
- n₄ is level mode (0 = no level mod; 1 = OOK; 2 = decrease output when “Low”)
- n₅ is number of dB to decrease output (+1 to +100) if n₄ = 2
- n₆ is cycles mode (0 = continuous; 1 = modulate for “number of cycles”)
- n₇...8 is a 16-bit integer (LSB first) = number of cycles (1-60,000) if n₆ = 1
- n₉ is square wave source (0 = internal; 1 = use aux input as mod source—aux input action must also be set to “Mod: Square wave source”)
- n₁₀ is frequency mode (0 = no freq mod; 1 = + offset; 2 = – offset)
- n₁₁...14 is a 32-bit integer (LSB first) representing the offset frequency in kHz if n₁₀ > 0
(1-4,365,000 but current frequency + offset cannot exceed range of 35 MHz to 4.4 GHz)
- n₁₅ is frequency scan mode (0 = no frequency scan; 1 = scan frequency) If set to 1, n₁₀ must be set to 0, and n₆ must be set to 0
- n₁₆...19 is a 32-bit integer (LSB first) representing the start scan frequency in kHz if n₁₅ = 1
(35,000 to 4,400,000)
- n₂₀...23 is a 32-bit integer (LSB first) representing the stop scan frequency in kHz if n₁₅ = 1
(35,000 to 4,400,000)
- n₂₄...27 is a 32-bit integer (LSB first) representing the frequency step in kHz if n₁₅ = 1
(1 to 1,000,000)
- n₂₈...29 is a 16-bit integer (LSB first) = number of microseconds to dwell at each frequency
(2-60,000)
- n₃₀ is frequency scan repetition (0 = continuous scan; 1 = one scan; 2 = number of scans)
- n₃₁...32 is a 16-bit integer (LSB first) = number of frequency scans when n₃₀ = 2
(2-60,000)

To set the square wave modulation parameters, send...

0x08, 0x26, n₀...n₃₂

...where n₀...n₃₂ are the modulation parameters as outlined above. Note that you must include all data bytes (n₀...n₃₂) even if not using all features. Also, all parameters MUST be legal values as indicated above.

The unit will respond with 2 bytes...

0x08, 0x26

2.39 Scan Parameters

To read the scan parameters, send...

0x07, 0x27

The unit will respond with 43 bytes...

0x07, 0x27, n0...n40

...where...

- n0 is the frequency function (0=none; 1=scan up; 2=scan down; 3=up/down; 4=down/up)
- n1...4 is a 32-bit unsigned integer (LSB first) representing the high limit scan frequency in kHz
- n5...8 is a 32-bit unsigned integer (LSB first) representing the low limit scan frequency in kHz
- n9 is the level function (0=none; 1=scan up; 2=scan down; 3=up/down; 4=down/up)
- n10...11 is a 16-bit signed integer (LSB first) representing the high level limit in dBm
- n12...13 is a 16-bit signed integer (LSB first) representing the low level limit in dBm
- n14 is the scan type (1=increment frequency; 2=increment level; 3=fit frequency; 4=fit level)
- n15...18 is a 32-bit unsigned integer (LSB first) representing the frequency increment in kHz
- n19 is an unsigned byte representing the level increment limit in dBm
- n20...23 is a 32-bit unsigned integer (LSB first) representing the frequency time increment in milliseconds
- n24...27 is a 32-bit unsigned integer (LSB first) representing the level time increment in milliseconds
- n28...31 is a 32-bit unsigned integer (LSB first) representing the total scan time in milliseconds
- n32 is the RF output auto on/off (0=don't automatically control output; 1=auto RF output on/off)
- n33...34 is a 16-bit unsigned integer (LSB first) representing the number of scans (0=continuous; 60,000 max)
- n35...36 is a 16-bit unsigned integer (LSB first) representing the inter-scan interval in ms (60,000 max)
- n37 controls inter-scan RF output muting (0=don't mute; 1=mute)
- n38 controls inter-scan frequency and level (0=jump to beginning-of-scan values; 1=hold end-of-scan values)
- n39 controls step level after each scan (0=don't step level; 1= step level after each scan)
- n40 controls repeat step level sequence (0=don't repeat; 1= repeat)

To set the scan parameters, send...

0x08, 0x27, n0...n40

...where n0...n40 are the scan parameters as outlined above. Note that you must include all data bytes (n0...n40) even if not using all features. Also, all parameters MUST be legal values as indicated above.

The unit will respond with 2 bytes...

0x08, 0x27

For insight into the meaning of some of the more unusual parameters listed above, review the “Scan tab” section of the user manual.

2.40 Control Script

The unit can store up to 201 command steps in its control script. The end-of-script marker occupies a step, so a maximum of 200 executable commands may be used. Each type of command is assigned a code number. Depending on the type of command, a parameter value may also be required. The parameter value must be set to zero for all commands that don't require an accompanying parameter. Following is the list of command codes and the range of acceptable values for each parameter.

Command Code	Description	Parameter Value	Parameter Units
0	End of script marker	0	
1	Turn RF output ON	0	
2	Turn RF output OFF	0	
3	Wait for command to continue	0	
4	Send beep command to host	0	
5	Set frequency	35.000 to 4400.000	MHz
6	Set power output level	-90 to +10	dBm
7	Pause	0.001 to 60.000	seconds
8	Trigger output ON	0	
9	Trigger output OFF	0	
10	Increment frequency	±0.001 to ±4365.000	MHz
11	Increment power	±1 to ±99	dB
12	Preset frequency ¹	1 to 12	
13	Preset power level ¹	1 to 12	
14	Script frequency ²	1 to 10	
15	Script power level ²	1 to 10	
16	Multiply frequency	0.100001 to 10	

¹ These refer to the presets numbered 1 to 12 for frequency and power level. They recall the value in the indicated preset and set the current frequency or power level to that value.

² These refer to the ten volatile frequency and ten volatile power level memories in the unit. They recall the value in the indicated memory and set the current frequency or power level to that value. See sections 2.40.10 and 2.40.12 to set these memories.

Each step in the script is assigned a step number, which is zero based, so the first step is step 0, and the highest possible step is step 200. The individual commands are sent to the unit, or read from the unit, one at a time. When a step is written, its step number must be included. Therefore, the steps don't necessarily need to be written in sequential order. The highest step number used contains the "End of script marker" or command code 0, and all lesser step numbers must be written with legitimate command codes and parameter values.

2.40.1 Read End of Script Step Number

To read the step number that contains the end-of-script marker, send...

0x07, 0x2A

The unit will respond with 3 bytes...

0x07, 0x2A, n

...where **n** equals the step number that was last written with the end-of-script marker. Note that this is also equal to the number of executable steps in the script. It is important to also note that attempting to read this value before the end-of-script marker has been written for a new script will result in reading the end-of-script step number from a previously written script. In other words, it is good practice to write all the steps of a new script followed by writing the end-of-script marker before attempting to read the step containing the end-of-script marker.

2.40.2 Write a Script Step

To write a script step to the unit, send...

0x08, 0x28, n₀...n₅

...where...

n₀ is the step number to write (0-200)

n₁ is the command code from the table to write (0-11)

n₂...5 is a 32-bit floating point value (IEEE-754, LSB first) = parameter value to write

The unit will respond with 2 bytes...

0x08, 0x28

2.40.3 Read a Script Step

To read a script step from the unit, send...

0x07, 0x28, n

...where n is the step number to read (0-200).

The unit will respond with 8 bytes...

0x07, 0x28, n₀...n₅

...where...

n₀ is the step number read (0-200)

n₁ is the command code for that step (0-11)

n₂...n₅ is a 32-bit floating point value (IEEE-754, LSB first) = parameter value for that step

The data returned from attempting to read a step number higher than the end-of-script step number is meaningless and should be ignored.

2.40.4 Write the Control Script Name

The control script may have a name attached to it. This name is displayed in TPI-Link when the script is read from the unit. The name is optional and may consist of up to 64 characters. To write a script name, send...

0x08, 0x29, n₀...n₆₄

...where...

n₀ is the number of characters in the script name (0-64)

n₁...n₆₄ is an ASCII string (not null terminated) = the script name

The unit will respond with 2 bytes...

0x08, 0x29

The number of bytes in the packet is dependent upon the length of the script name. To erase the name send...

0x08, 0x29, 0x00

2.40.5 Read the Control Script Name

To read the control script name, send...

0x07, 0x29

The unit will respond with 3 to 67 bytes...

0x07, 0x29, n₀...n₆₄

...where...

n₀ is the number of characters in the scrip name (0-64)

n₁...n₆₄ is an ASCII string (not null terminated) = the script name

2.40.6 Set the Control Script Run Control

To set the control script run control, send...

0x08, 0x2B, n₀, n₁

...where...

n ₀	Unit State	n ₁
0x00	Repeat script continuously	1-100 (ignored)
0x01	Run script once	1-100 (ignored)
0x02	Run script a number of times	1-100*

*Number of times to run script

The unit will respond with 2 bytes...

0x08, 0x2B

This command does not actually start the script running. It only specifies how many times the script will run when it is started. To start or stop the script running see section 2.8.

2.40.7 Read the Control Script Run Control

To read the control script run control, send...

0x07, 0x2B

The unit will respond with 4 bytes...

0x07, 0x2B, n₀, n₁

...where...

n ₀	Unit State	n ₁
0x00	Repeat script continuously	Ignore value
0x01	Run script once	Ignore value
0x02	Run script a number of times	1-100*

*Number of times to run script

2.40.8 Current Control Script Command

When each step of the command script is executed, the unit will send...

0x07, 0x2C, n₀...n₅

...where...

n₀ is the step number (0-200)

n₁ is the command code for that step (0-11)

n₂...n₅ is a 32-bit floating point value (IEEE-754, LSB first) = parameter value for that step

This is never requested by the user. It is automatically sent during script execution so that the user will know the current position of the executing script. Note that it is only sent during the first run of the script. If the run control is set for continuous running or to run a specific number of times, this packet will only be sent for each step of the first run of the script. The exception to this is that during all subsequent runs of the script, it will also be sent for each “wait” command step and each “beep” command step.

2.40.9 Read Control Script Frequency

The unit contains ten control script frequency memories that are volatile. They will all revert to 35 MHz each time the unit powers up. These memories may be written and read with the following two commands. Control script command code 14 is used within the script to set the current RF output frequency to the value in one of the ten memories.

To read the control script frequencies, send...

`0x07, 0x2D, n`

...where `n` is the number of the memory (1 through 10) that you wish to read.

The unit will respond with 7 bytes...

`0x07, 0x2D, n0...n4`

...where...

`n0` is the number of the memory (1-10)

`n1...4` is a 32-bit integer (LSB first) representing the frequency in kHz.

2.40.10 Set Control Script Frequency

To set the control script frequencies, send...

`0x08, 0x2D, n0...n4`

...where...

`n0` is the number of the memory (1-10)

`n1...n3` is a 32-bit integer (LSB first) representing the frequency in kHz. The value must be between 35000 and 4400000 inclusive.

The unit will respond with 2 bytes...

`0x08, 0x2D`

2.40.11 Read Control Script Power Level

The unit contains ten control script power level memories that are volatile. They will all revert to 0 dBm each time the unit powers up. These memories may be written and read with the following two commands. Control script command code 15 is used within the script to set the current RF output power level to the value in one of the ten memories.

To read the control script power levels, send...

0x07, 0x2E, n

...where n is the number of the memory (1 through 10) that you wish to read.

The unit will respond with 4 bytes...

0x07, 0x2E, n₀...n₁

...where...

n₀ is the number of the memory (1-10)

n₁ is a signed byte representing the output level in dBm.

2.40.12 Set Control Script Power Level

To set the control script power levels, send...

0x08, 0x2E, n₀...n₁

...where...

n₀ is the number of the memory (1-10)

n₁ is a signed byte representing the output level in dBm.

The unit will respond with 2 bytes...

0x08, 0x2E

2.41 Read ADF4351 Registers

To read the current state of the ADF4351 registers, send...

0x07, 0x2F

The unit will respond with 26 bytes...

0x07, 0x2F, n₀...n₂₃

...where...

n₀...n₃ is a 32-bit integer (low byte first) representing the contents of register 0

n₄...n₇ is a 32-bit integer (low byte first) representing the contents of register 1

n₈...n₁₁ is a 32-bit integer (low byte first) representing the contents of register 2

n₁₂...n₁₅ is a 32-bit integer (low byte first) representing the contents of register 3

n₁₆...n₁₉ is a 32-bit integer (low byte first) representing the contents of register 4

n₂₀...n₂₃ is a 32-bit integer (low byte first) representing the contents of register 5

2.42 Set ADF4351 Registers

To set the contents of the ADF4351 registers, send...

0x08, 0x2F, $n_0 \dots n_{24}$

The individual bits of n_0 determine what registers to write.

n_0	Unit State
bits 7:6	Reserved (always write 0)
bit 5	1 = write register 5
bit 4	1 = write register 4
bit 3	1 = write register 3
bit 2	1 = write register 2
bit 1	1 = write register 1
bit 0	1 = write register 0

Bytes $n_1 \dots n_{24}$ contain the data to write:

$n_1 \dots n_4$ is a 32-bit integer (low byte first) representing the contents of register 0

$n_5 \dots n_8$ is a 32-bit integer (low byte first) representing the contents of register 1

$n_9 \dots n_{12}$ is a 32-bit integer (low byte first) representing the contents of register 2

$n_{13} \dots n_{16}$ is a 32-bit integer (low byte first) representing the contents of register 3

$n_{17} \dots n_{20}$ is a 32-bit integer (low byte first) representing the contents of register 4

$n_{21} \dots n_{24}$ is a 32-bit integer (low byte first) representing the contents of register 5

All 25 bytes of data must always be sent, even if you are not changing a register. For registers not changing, i.e., their corresponding n_0 bit is zero, the data will be ignored, so it may be any value. The registers to be written will be written in reverse order, i.e., register 5 will be written first, and register 0 will be written last.

The unit will respond with 2 bytes...

0x08, 0x2F

Do not attempt to write to the registers until you thoroughly understand the function of each bit of each register, or unpredicted functioning may result. Refer to the ADF4351 data sheet.

2.43 VCO Control

To read the VCO control state, send...

0x07, 0x30

The unit will respond with 3 bytes...

0x07, 0x30, n

...where **n = 0** means the VCO is always powered, and **n = 1** means the VCO is powered down when the RF output is muted.

To set the VCO control state, send...

0x08, 0x30, n

...where **n = 0** means you wish the VCO to remain powered, and **n = 1** means you wish to power down the VCO when the RF output is muted.

The unit will respond with 2 bytes...

0x08, 0x30

2.44 Auxiliary Level Control (TPI-1001 & TPI-1005 only)

To read the auxiliary input level control parameters, send...

0x07, 0x31

The unit will respond with 10 bytes...

0x07, 0x31, n0...n7

...where...

n0...n3 is a 32-bit floating point value (IEEE-754, LSB first) = voltage that equals 0 dBm

n4...n7 is a 32-bit floating point value (IEEE-754, LSB first) = dB/volt^{Note 2}

To set the auxiliary input level control parameters, send...

0x08, 0x31, n0...n7

...where...

n0...n3 is a 32-bit floating point value (IEEE-754, LSB first) = voltage that equals 0 dBm

n4...n7 is a 32-bit floating point value (IEEE-754, LSB first) = dB/volt^{Note 2}

The unit will respond with 2 bytes...

0x08, 0x31

Notes:

¹ If this command is sent to a TPI-1002, an error code will be triggered because the TPI-1002 does not have an auxiliary input.

² A negative dB/volt value will decrease level as the voltage increases.

2.45 Auxiliary Frequency Control (TPI-1001 & TPI-1005 only)

To read the auxiliary input frequency control parameters, send...

0x07, 0x32

The unit will respond with 11 bytes...

0x07, 0x32, n₀...n₈

...where...

n₀...n₃ is a 32-bit floating point value (IEEE-754, LSB first) = frequency in MHz at zero volts

n₄...n₇ is a 32-bit floating point value (IEEE-754, LSB first) = MHz/volt ^{Note 2}

n₈ is an unsigned integer = milliseconds between frequency changes (range is 2-100)

To set the auxiliary input frequency control parameters, send...

0x08, 0x32, n₀...n₈

...where...

n₀...n₃ is a 32-bit floating point value (IEEE-754, LSB first) = frequency in MHz at zero volts

n₄...n₇ is a 32-bit floating point value (IEEE-754, LSB first) = MHz/volt ^{Note 2}

n₈ is an unsigned integer = milliseconds between frequency changes (range is 2-100)

The unit will respond with 2 bytes...

0x08, 0x32

Notes:

¹ If this command is sent to a TPI-1002, an error code will be triggered because the TPI-1002 does not have an auxiliary input.

² A negative MHz/volt value will decrease frequency as the voltage increases.

2.46 Frequency List (TPI-1001 & TPI-1005 only)

This is a feature in the TPI-1001 & TPI-1005. You can program up to 1000 random frequencies in non-volatile memory. You then can step through the list of frequencies by applying a signal to the auxiliary input. The list of frequencies may contain from 1 to 1000 frequencies although a list of less than two frequencies is not useful. After the last frequency in the list, you need to program an end-of-list marker in the next memory position so that the unit knows when to stop advancing to the next frequency. If memory position 1000 is used, you do not need to (cannot) program an end-of-list marker because there is no memory position 1001.

To set a frequency in the list, send...

0x08, 0x35, n₀...n₅

...where...

n₀...n₁ is a 16-bit unsigned integer (LSB first) = memory position (range is 1-1000)

n₂...n₅ is a 32-bit unsigned integer (LSB first) = frequency in kHz (range is 35,000 to 4,400,000)

If zero is written for the frequency, this indicates an end-of-list marker.

If a number from 1 to 1000 is written for the frequency, this indicates a jump to that position.

The unit will respond with 2 bytes...

0x08, 0x35

To read a frequency from the list, send...

0x07, 0x35, n₀...n₁

...where...

n₀...n₁ is a 16-bit unsigned integer (LSB first) = memory position (range is 1-1000)

The unit will respond with 6 bytes...

0x07, 0x35, n₀...n₃

...where...

n₀...n₃ is a 32-bit unsigned integer (LSB first) = frequency in kHz (range is 35,000 to 4,400,000)

If zero is returned, this memory position contains an end-of-list marker.

If a number from 1 to 1000 is returned, this memory position contains a jump to that position.

Before you begin stepping through the frequency list, you need to set a starting point in the list. In other words, you need to set the current frequency list position.

To set the position in the list, send...

0x08, 0x36, n₀...n₁

...where...

$n_0 \dots n_1$ is a 16-bit unsigned integer (LSB first) = memory position (range is 1-1000)

The unit will respond with 2 bytes...

0x08, 0x36

To read the current position in the list, send...

0x07, 0x36

The unit will respond with 4 bytes...

0x07, 0x36, $n_0 \dots n_1$

...where...

$n_0 \dots n_1$ is a 16-bit unsigned integer (LSB first) = current memory position (range is 1-1000)

Note that it is possible to have several lists of frequencies stored one after the other with an End Marker following each list. Set the position to the first frequency in the list you wish to execute. The execution will stop when an end marker is encountered. If jump is encountered, execution will jump to the frequency stored in the position jumped to.

For the auxiliary input to function as a signal to advance to the next frequency in the list, you must set the auxiliary input action to “Next frequency in list”. See section 2.19.

2.47 FM Source (TPI-1005 only)

To read the FM Source, send...

0x07, 0x37

The unit will respond with 3 bytes...

0x07, 0x37, n

...where...

n	FM Source
0x00	FM Input
0x01	Waveform Output

To set the FM Source, send...

0x08, 0x37, n

The unit will respond with 2 bytes...

0x08, 0x37

Note: The previously saved value (see section 2.32) will be restored when user mode is enabled (see section 2.1)

2.48 FM Method (TPI-1005 only)

To control the FM method for the TPI-1006, see section 2.55.

To read the FM Method, send...

0x07, 0x38

The unit will respond with 3 bytes...

0x07, 0x38, n

...where...

n	FM Method
0x00	No FM
0x01	Modulate Loop Filter
0x02	Modulate the VCTCXO

To set the FM Method, send...

0x08, 0x38, n

The unit will respond with 2 bytes...

0x08, 0x38

Note: The previously saved value (see section 2.32) will be restored when user mode is enabled (see section 2.1)

2.49 Amplifier Source (TPI-1005 only)

To read the Amplifier Source, send...

0x07, 0x39

The unit will respond with 3 bytes...

0x07, 0x39, n

...where...

n	Amplifier Source
0x00	DTMF Generator
0x01	Waveform Generator
0x02	Ramp Generator
0x03	FM Input

To set the Amplifier Source, send...

0x08, 0x39, n

The unit will respond with 2 bytes...

0x08, 0x39

Note: The previously saved value (see section 2.32) will be restored when user mode is enabled (see section 2.1)

2.50 Waveform Generator (TPI-1005 only)

To read the Waveform Generator settings, send...

0x07, 0x3A

The unit will respond with 8 bytes...

0x07, 0x3A, n0-n5

...where...

n0...n3 is a 32-bit floating point value (IEEE-754, LSB first) = the frequency in Hz

(Sine: 20.0 Hz to 20,000.0 Hz; Triangle: 20.0 Hz to 1,000.0 Hz; Square: 20.0 Hz to 2,000.0 Hz)

n4 is a byte = wave shape (0 = sine wave; 1 = triangle wave, 2 = square wave)

n5 is a byte = waveform generator state (0 = off, 1 = on, 2 = gated by Beacon)

To set the Waveform Generator settings, send...

0x08, 0x3A, n0-n5

The unit will respond with 2 bytes...

0x08, 0x3A

2.51 Amplifier Gain (TPI-1005 only)

To read the Amplifier Gain, send...

0x07, 0x3B

The unit will respond with 3 bytes...

0x07, 0x3B, n

...where **n** equals the gain (range: 0 to 255). Note: The gain is uncalibrated and is a linear potentiometer on the amplifier input.

To set the Amplifier Gain, send...

0x08, 0x3B, n

The unit will respond with 2 bytes...

0x08, 0x3B

2.52 Analyzer (TPI-1001 and TPI-1005 only)

Beginning with firmware version 1.066 there are user commands to control the scalar network analyzer of the TPI-1001 and TPI-1005.

The analyzer is first programmed with various parameters including start frequency, stop frequency, etc. Then the analyzer is “controlled” by sending a command to start it. The control command includes a parameter to specify the maximum interval between data packets to be received during a scan. The data is then sent in packets of limited size until the entire scan is completed.

2.52.1 Analyzer Parameters

To read the analyzer parameters, send...

0x07, 0x3C

The unit will respond with 23 bytes...

0x07, 0x3C, n0...n20

...where...

n0...3 is a 32-bit unsigned integer (LSB first) representing the start frequency in kHz

n4...7 is a 32-bit unsigned integer (LSB first) representing the stop frequency in kHz

n8...11 is a 32-bit unsigned integer (LSB first) representing the step frequency in kHz

n12...13 is a 16-bit unsigned integer (LSB first) representing the dwell time per point in ms (2-500)

n14...17 is a 32-bit unsigned integer (LSB first) representing the number of points to measure

n18 controls auto RF on/off (0=don't change RF output; 1=RF output on during acquisition)

n19 is an unsigned byte = the maximum number of points to report per packet (max 50)

n20 is an unsigned byte = no. of detector readings to average per point (2-10, 0=one reading per point)

To set the analyzer parameters, send...

0x08, 0x3C, n0...n20

...where n0...n20 are the analyzer parameters as outlined above. Note that you must include all data bytes (n0...n20) even if not using all features. Also, all parameters MUST be legal values as indicated above.

The unit will respond with 2 bytes...

0x08, 0x3C

It may seem redundant to specify the number of points to measure as that information can be surmised from the stop frequency and the step frequency. However, it is up to the user to calculate this and supply the total number of points to be measured. Note that, for example, a scan from 100 MHz to 200 MHz with a step frequency of 1 MHz will measure 101 points.

2.52.2 Control Analyzer

To control the analyzer, send...

0x08, 0x3D, n₀...n₄

...where...

n₀ controls the state of the analyzer (1=start; 0=stop)

n₁ selects sweeps (0=single sweep; 1=continuous sweeps)

n₂...n₃ is a 16-bit unsigned integer (LSB first) sets the maximum milliseconds between data packets

n₄ selects auxiliary input control (0=normal operation; 1=arm analyzer start—aux input starts)

Note that you must include all data bytes (n₀...n₄) even if not using all features. Also, all parameters MUST be legal values as indicated above.

The unit will respond with 2 bytes...

0x08, 0x3D

The maximum number of ms between data packets is specified to ensure that a continuous stream of data is received for display purposes. For example, if the unit is taking a long time to build a packet because the dwell on each point is large, you can specify that a packet is received more frequently even if it does not contain very many data points. This keeps the plot line on the display building. A larger value will create larger and fewer packets, which can be more efficient. Note that when the maximum number of points per packet has been read, the packet will be sent even if it is before the “milliseconds between packets” interval has elapsed as specified in analyzer parameters.

2.52.3 Analyzer Data

While the analyzer is running, the unit will send data packets with variable number of bytes. These are not requested. The packet format is:

0x07, 0x3E, n₀...n_?

...where...

n₀ is an unsigned byte = the number of data points in this packet

n_{1...4} is a 32-bit unsigned integer (LSB first) = scan step of first data point (0 based)

n_{5...?} data points are multiple 32-bit floating-point values (IEEE-754, LSB first) in dBm.

For example, if the number of data points reported is 30, and the scan step of the first data point is 45, then the first data point in the packet will occupy the 46th point because the value is zero based. This packet would contain the data for scan points 46 through 75. The next packet would start with scan point 76, etc.

2.52.4 Analyzer Stopped

When the analyzer has stopped, either because it was commanded to or because it reached the end of a single scan, the following packet will be sent.

0x07, 0x3F

2.53 Trigger Output (TPI-1001 & TPI-1005 only)

The trigger output on the TPI-1001 and TPI-1005 may be directly controlled by a command provided the trigger output source is set to “none”. See section 2.20.

To control the trigger output, send...

`0x08, 0x40, n`

...where...

`n` controls the trigger output (`n=1` asserts¹ the output, `n=0` deasserts the output)

¹ Asserting the output makes it high (≈ 3.3 volts) unless the trigger output is inverted. See section 2.21.

2.54 ADC Average

The analog-to-digital converter (ADC) in the unit is used to read the supply voltages, and, in the case of units with an RF detector, is used to read the RF input level. Beginning with firmware version 1.069 the ADC can be programmed to read more than one analog level at a time and average them. This has the potential to reduce noise in the signal.

To read the number of ADC conversions to average, send...

0x07, 0x41

The unit will respond with 3 bytes...

0x07, 0x41, n

...where n equals the number of conversions to average (range: 1 to 8).

To set the number of ADC conversions to average, send...

0x08, 0x41, n

The unit will respond with 2 bytes...

0x08, 0x41

If the value of n sent is 0, the unit will use 1 conversion. If the value of n sent is greater than 8, the unit will use 8 conversions.

2.55 FM Method (TPI-1006 only)

To control the FM method for the TPI-1005, see section 2.48.

The TPI-1006 unit has the ability for the FM input to modulate either the PLL loop filter or the voltage controlled temperature compensated crystal oscillator (VCTCXO).

To read the FM method, send...

0x07, 0x42

The unit will respond with 3 bytes...

0x07, 0x42, n

...where...

n	FM Method
0x00	Loop filter
0x01	VCTCXO

To select the FM method, send...

0x08, 0x42, n

The unit will respond with 2 bytes...

0x08, 0x42

2.56 Error Notification

If an error is encountered, the unit will send...

0x07, 0xFF, n

...where n is the error number (decimal) according to the following table...

Error #	Description
1	Checksum error
2	Undefined command type (first body byte)
3	Undefined command (second body byte)
4	Data out of range
5	Improper register ID
6	Illegal beacon message character
7	Requested RF level < -90 dBm
8	Internal beacon message length error
9	Unknown script command
10	No detector available
11	No auxiliary input available
12	No trigger output available
27	Communication watchdog timeout
88	Failed to write EEPROM
89	Failed to read EEPROM

3 C# Code Examples

This section will give some C# sample code snippets that can be referenced by a programmer to develop a program that communicates with the unit.

3.1 Establish the COM Port

Establish the COM port...

```
using System.IO;
using System.IO.Ports;

public partial class Form1 : Form {
    private SerialPort sp = new SerialPort();
}

public Form1() {
    InitializeComponent();
    sp.DataReceived += new SerialDataReceivedEventHandler(sp_DataReceived);
}
```

Create a function similar to this to open the port. This function will return “true” if the port was successfully opened.

```
private Boolean Open_Port() {
    try {
        sp.PortName = "COM4"; // Substitute your COM port number for "COM4"
        sp.Open();
        if (sp.IsOpen) {
            sp.BaudRate = 3000000;
            sp.DataBits = 8;
            sp.StopBits = StopBits.One;
            sp.Parity = Parity.None;
            sp.Handshake = Handshake.RequestToSend;
            sp.Encoding = System.Text.Encoding.GetEncoding(1252);
            return true;
        }
        else {
            return false;
        }
    }
    catch {
        return false;
    }
}
```

3.2 Send a Packet

As an example, send this packet to enable user control. The “7” is the number of bytes to send.

```
sp.Write(new byte[] { 0xAA, 0x55, 0x00, 0x02, 0x08, 0x01, 0xF4 }, 0, 7);
```

3.3 Read Bytes From the COM Port

The following code is an example of how to read bytes from the COM port.

```
private void sp_DataReceived(object sender, SerialDataReceivedEventArgs e) {  
    byte z;  
    while (sp.IsOpen && sp.BytesToRead > 0) {  
        z = (byte) sp.ReadByte(); // Get next byte from serial port  
        // Do something here with the bytes received  
        // This code will run for each byte received  
    }  
}
```

The programmer needs to devise a method of parsing the bytes received to establish the beginning of a packet, its length, and if the checksum is correct. The data from the packet may then be examined to understand the meaning of the packet.

4 ASCII Commands

As an alternative to the preceding discussion, beginning with firmware version 1.065, the unit can accept simple ASCII strings to control the basic parameters.

ASCII control must first be enabled in the unit. This is accomplished via TPI-Link by checking “Enable ASCII Control” in the lower left corner of the System tab. See Figure 1.

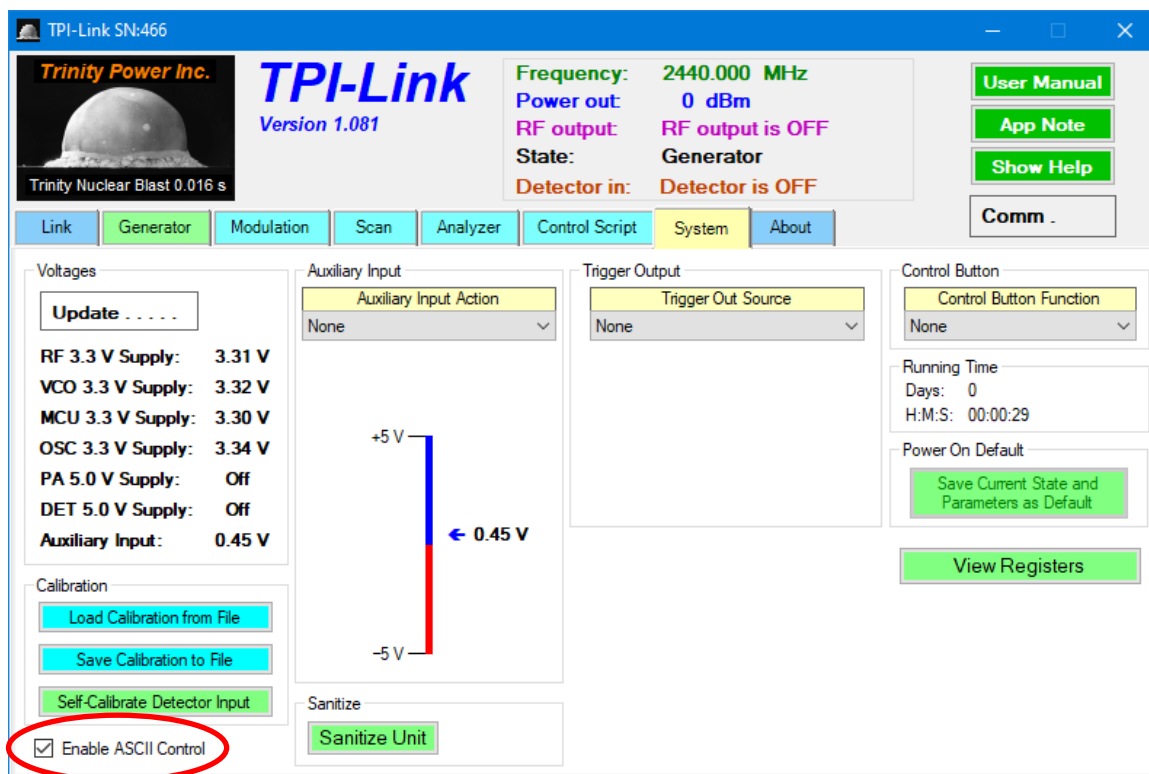


Figure 1. Enable ASCII Control

The unit will remember that ASCII has been enabled. TPI-Link can still operate with ASCII control enabled.

All ASCII commands must begin with the number sign, #, followed by a few characters that describe the parameter, followed by an equal sign, =, followed by a value, followed by a Line Feed character (0x0A). Spaces may be inserted as desired and are ignored. Carriage Return characters (0x0D) are also ignored. The parameter names are not case sensitive.

A terminal program is an easy way to test the following commands.

4.1 RF output on/off

To turn the RF output on, send: “#RF=ON”. The unit will respond with “RF = ON”.

To turn the RF output off, send: “#RF=OFF”. The unit will respond with “RF = OFF”.

To query the unit about the RF output state, send: “#RF=?”.

The unit will respond “RF = ON” or “RF = OFF”.

4.2 Frequency

To set the RF frequency, send: “#FREQ=n.nnn”, where n.nnn is the desired frequency in MHz with 0-3 decimal places. Do not use more than three decimal places. The frequency resolution of the unit is 1 kHz. If n is less than the minimum operating frequency, the frequency will be set to the minimum. If n is greater than the maximum operating frequency, the frequency will be set to the maximum.

The unit will respond with “FREQ = n.nnn MHz”.

To query the current frequency, send: “#FREQ=?”.

4.3 Output level

To set the RF output level, send: “#LEVEL=n”, where n is the desired output level in dBm with no decimal places. The output level resolution is 1 dB. If n is less than the unit’s minimum level, the level will be set to the minimum level. Do not attempt to set the level less than -100. If n is greater than the unit’s maximum level, the level will be set to the maximum level. Do not attempt to set the level greater than 100.

Then unit will respond with “LEVEL = n dBm”.

To query the current level, send: “#LEVEL=?”.

4.4 Detector level

TPI-1001 and TPI-1005 have RF level detector inputs. To read the RF detector level, send: “#DET=?”. The unit will respond with the detector level in dBm to one decimal resolution.

To query the current detector level, send: “#DET=?”.

5 SCPI Commands

Beginning with firmware version 1.074, a limited set of SCPI commands were added. SCPI stands for Standard Commands for Programmable Instruments. Before the unit will respond to SCPI, the feature must be enabled. Link to the unit with TPI-Link and go to the System tab. Check the “SCPI Control” box. See Figure 2.

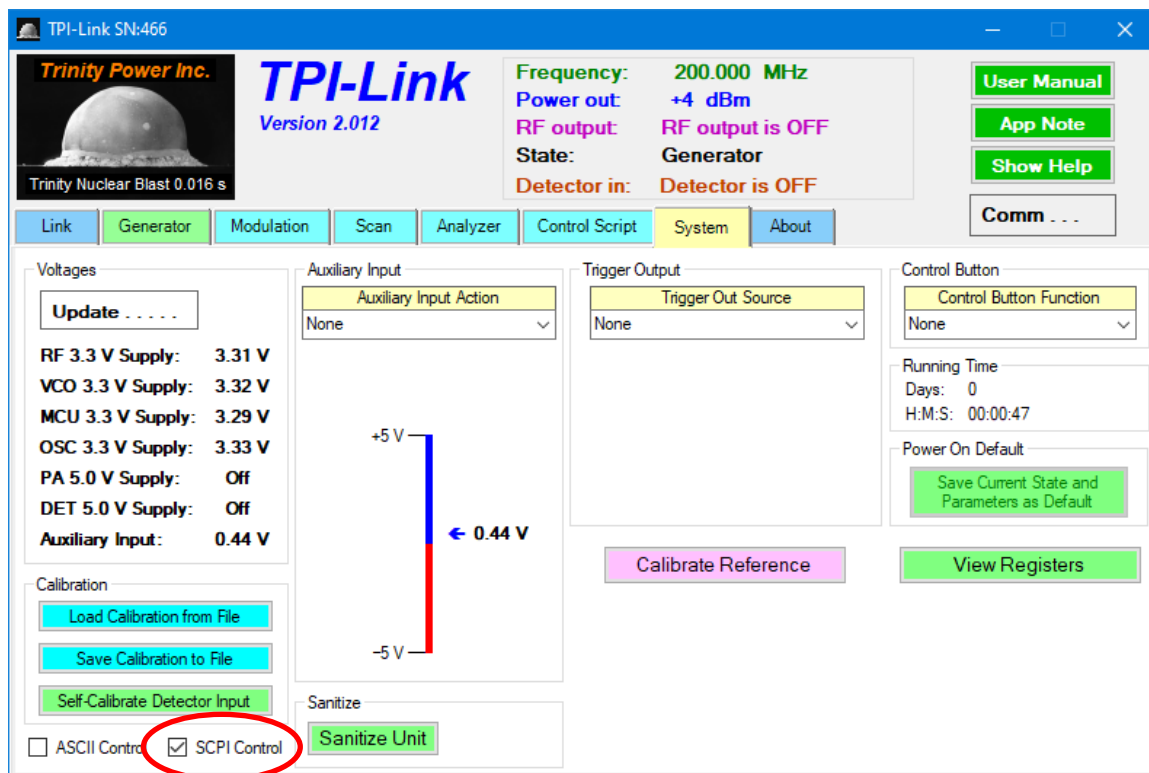


Figure 2. Enable SCPI Control

It's a good idea to make sure that “ASCII Control” is NOT checked. TPI-Link can now be closed. Connect to the unit using the COM port that was identified on the Link tab using your own software.

The unit will remember that SCPI has been enabled. TPI-Link can still operate with SCPI control enabled.

The following SCPI commands are available.

5.1 *RST

Sending *RST will force the unit to exit any special mode of operation and return to simple generator mode. It will also ensure that the RF output is turned off.

5.2 [:]SOURce:FREQuency <num>

The number must be the desired frequency in MHz with a maximum of three decimal places. The resolution of the unit is 1 kHz. If the value is below the unit's minimum frequency, the minimum frequency will be selected. If the value is above the unit's maximum frequency, the maximum frequency will be selected.

5.3 [:]SOURce:FREQuency?

This command queries the current frequency. The result will be in MHz with a “MHz” suffix.

5.4 [:]SOURce:POWer <integer>

The integer is the desired RF output power level in dBm with no decimal places. The output level resolution is 1 dB. If the integer is less than the unit’s minimum level, the level will be set to the minimum level. Do not attempt to set the level less than -100. If the integer is greater than the unit’s maximum level, the level will be set to the maximum level. Do not attempt to set the level greater than 100.

5.5 [:]SOURce:POWer?

This command queries the current output power level. The result will be in dBm with a “dBm” suffix.

5.6 [:]SOURce:OUTPut:STATe ON|OFF

This command turns the RF output on/off.

5.7 [:]SOURce:OUTPut:STATe?

This command queries the current state of the RF output. The unit will return either ON or OFF.

6 Revisions

Rev 1.0	September 23, 2015	Original version supporting unit firmware version 1.017
Rev 1.1	October 17, 2015	Added section 3 with programming examples
Rev 1.2	November 28, 2015	Noted section 2.13 as “TPI-1001 only” Fixed typo in section 3.1
Rev 1.3	February 23, 2016	Added sections 2.35 and 2.36 about PLL lock Corrected hex equivalent in section 1.2
Rev 1.4	April 4, 2016	Added section 2.37 about running time
Rev 1.5	April 21, 2016	Added section 0 about square wave modulation parameters
Rev 1.6	July 22, 2016	Added section 0 about scan parameters
Rev 1.7	August 22, 2016	Added section 2.40 about control scripts
Rev 1.8	October 20, 2017	Section 2.40—added command codes 12 through 15 Added section 2.40.9—read control script frequency Added section 2.40.10—set control script frequency Added section 2.40.11—read control script power level Added section 2.40.12—set control script power level
Rev 1.9	March 4, 2018	Section 2.40—added command code 16
Rev 1.10	July 4, 2018	Added section headers to pdf bookmarks
Rev 1.11	August 19, 2018	Added sections 2.41 and 2.42 about ADF4351 registers
Rev 1.12	August 22, 2018	Fixed typo in section 0
Rev 1.13	September 18, 2018	Section 2.20—added trigger output sources
Rev 1.14	January 27, 2019	Section 2.38—added frequency scan
Rev 1.15	March 26, 2019	Sections 2.22 & 2.30—added 6 more frequency presets Sections 2.23 & 2.31—added 6 more level presets Section 2.38—reduced minimum on/off times
Rev 1.16	April 10, 2021	Added section 2.43 about VCO control
Rev 1.17	July 10, 2021	Section 2.19—added 3 auxiliary input actions Added section 2.44 about auxiliary level control Added section 2.45 about auxiliary frequency control
Rev 1.18	January 25, 2023	Section 2.19—added “Next frequency in list” Added section 2.46 “Frequency List”

Rev 1.19	December 6, 2023	Added section 2.47 “FM Source” Added section 2.48 “FM Method” Added section 2.49 “Amplifier Source” Added section 2.50 “Waveform Generator” Added section 2.51 “Amplifier Gain”
Rev 1.20	December 9, 2023	Added section 4 “ASCII Commands”
Rev 1.21	December 16, 2023	Added section 2.52 “Analyzer” Added section 2.53 “Trigger Output”
Rev 1.22	May 6, 2024	Section 2.52.3—Clarified data point units Added section 2.54 “ADC Average”
Rev 1.23	June 26, 2024	Section 2.46—Added jump capability
Rev 1.24	October 7, 2024	Added section 4.4 “Detector level”
Rev 1.25	November 5, 2024	Added section 5 “SCPI Commands”
Rev 1.26	November 15, 2024	Section 2.52.1—Changed “points to measure” to 32-bit integer Section 2.52.3—Changed “scan step” number to 32-bit integer
Rev 1.27	June 29, 2025	Section 2.50—Corrected typo Added section 2.55 “FM Method (TPI-1006 only)”